LAB 6: CODE AND BRANCH COVERAGE ANALYSIS

Course: Software Engineering
Experiment: Code and Branch Coverage Analysis
Student Name: Shreya Narayana Bhat
SRN: PES2UG24CS483

1. OBJECTIVE

The objective of this experiment is to analyze code coverage and branch coverage for a Python application using the `pytest` and `coverage` tools. The experiment aims to identify untested statements and decision branches, design additional test cases, and achieve 100% code coverage and 100% branch coverage.

2. PROBLEM DESCRIPTION

The application, `scholarship_awards.py`, implements scholarship-related functionalities. It contains three main functions:

1. `award_amount()` – Calculates the scholarship award amount based on the applicant's category, marks, income level, and renewal status.
2. `is_eligible()` – Determines scholarship eligibility based on attendance, backlogs, and special quota status.
3. `application_status()` – Determines whether an application is open, closing, or closed based on the remaining days, renewal status, and paper application status.

The objective is to develop comprehensive test cases that execute the statements and decision branches within these functions.

3. TOOLS AND TECHNOLOGIES USED

- **Programming Language:** Python
- **Testing Framework:** pytest
- **Coverage Tool:** coverage.py
- **Version Control:** Git and GitHub
- **Development Environment:** Visual Studio Code
- **Operating System:** Windows

4. CODE COVERAGE ANALYSIS

4.1 Definition

Code coverage measures the percentage of executable statements in a program that are executed when the test cases are run.

It helps identify statements that have not been tested.

The formula for statement coverage is:

Statement Coverage (%) = (Number of Executed Statements / Total Executable Statements) × 100

4.2 Implementation

A test file named `CodeCoverageTest_scholarship_awards.py` was created using pytest.

The test cases cover:

- Marks below the scholarship cutoff.
- Scholarship calculations for low-income and high-income applicants.
- Large scholarship awards.
- Renewal-related cutoff changes.
- Different scholarship categories.
- Invalid categories and negative marks.
- Regular and special-quota eligibility.
- Attendance and backlog conditions.
- Open, closing, and closed application statuses.
- The tests were executed using the following commands:

```
python -m coverage erase
python -m coverage run --source=scholarship_awards -m pytest
CodeCoverageTest_scholarship_awards.py
python -m coverage report -m
```

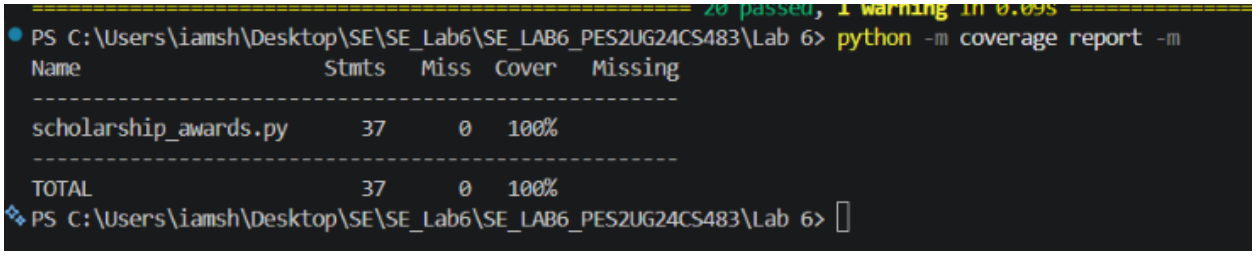
4.3 Results

The code coverage analysis produced the following results:

Metric	Result
Total executable statements	37
Missed statements	0
Statement coverage	100%

All 37 executable statements were executed during testing. No statements remained uncovered.

4.4 Screenshot



5. BRANCH COVERAGE ANALYSIS

5.1 Definition

Branch coverage measures the percentage of possible decision outcomes that are executed by the test cases.

For example, an `if-else` statement contains different possible execution paths. Branch coverage checks whether these decision outcomes have been exercised.

The formula for branch coverage is:

Branch Coverage (%) = (Number of Executed Branches / Total Branches) × 100

Branch coverage provides additional testing information beyond statement coverage because executing a statement does not necessarily mean that every possible decision outcome has been tested.

5.2 Implementation

A separate test file named `BranchCoverageTest_scholarship_awards.py` was created to test the different decision outcomes in the application.

The test cases cover:

- Valid and invalid scholarship categories.
- Negative marks and marks at scholarship cutoffs.
- Renewal and non-renewal conditions.
- Low-income and other income categories.
- Award calculations for different marks ranges.
- Regular eligibility and special-quota eligibility.
- Attendance and backlog boundary conditions.
- Open, closing, and closed application statuses.
- Different application cutoffs for renewals and paper applications.

The tests were executed using the following commands:

```
python -m coverage erase
python -m coverage run --branch --source=scholarship_awards -m pytest
BranchCoverageTest_scholarship_awards.py
python -m coverage report -m
```

The `--branch` option enables branch coverage measurement in addition to statement coverage.

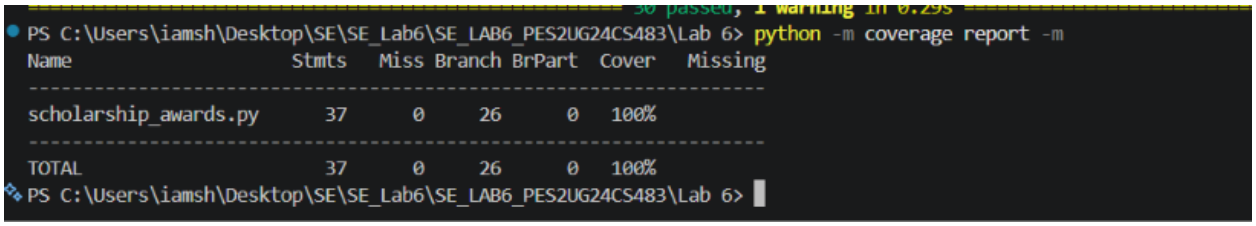
5.3 Results

The branch coverage analysis produced the following results:

Metric	Result
Total executable statements	37
Missed statements	0
Total branches	26
Partial branches	0
Overall coverage	100%

All 37 executable statements and all 26 reported branches were covered. No partial branches remained.

5.4 Screenshot



6. VERSION CONTROL USING GIT AND GITHUB

Git was used to maintain version history and track the testing work. The project was uploaded to a personal GitHub repository.

Separate commits were created for the code coverage tests and branch coverage tests. Additional commits were made for screenshot filename correction and exclusion of generated coverage and cache files.

The final changes were successfully pushed to the `main` branch of the GitHub repository.

GitHub Repository:
https://github.com/Shreya-Bhat-2006/SE_LAB6_PES2UG24CS483

7. COMPARISON OF CODE COVERAGE AND BRANCH COVERAGE

Aspect	Code Coverage	Branch Coverage
Purpose	Measures executed statements	Measures executed decision branches
Focus	Statement execution	Decision outcomes
Coverage option	`coverage run`	`coverage run --branch`
Test file	`CodeCoverageTest_scholarship_awards.py`	`BranchCoverageTest_scholarship_awards.py`
Result	100% statement coverage	100% statement and branch coverage

Branch coverage provides a more detailed view of decision testing because it verifies that the different outcomes of conditional statements have been exercised.

8. CONCLUSION

In this experiment, code coverage and branch coverage analysis were performed on the scholarship awards application using Python, pytest, and coverage.py.

The initial testing process was extended with additional test cases to cover the application's statements and decision branches. The final results achieved 100% statement coverage and 100% branch coverage, with all 37 executable statements and all 26 reported branches covered.

This experiment demonstrates the importance of systematic testing in identifying untested code and improving confidence in software behavior. Git and GitHub were also used to maintain version history and submit the completed work.

Final Result: Successfully achieved 100% code coverage and 100% branch coverage for the scholarship awards application.